

Does Latent Planning Survive Point Clouds? Action-Conditioned JEPA World Models for Geometric Observations

Fabio F. Oberweger*

AIT Austrian Institute of Technology
Assistive & Autonomous Systems
fabio.oberweger@ait.ac.at

Michael Schwingshackl*

AIT Austrian Institute of Technology
Assistive & Autonomous Systems
michael.schwingshackl@ait.ac.at

 Website

 Code

Abstract

JEPA world models make latent-space planning a practical route to control, but they are built almost exclusively on images. Whether latent prediction survives geometric observations is unclear: point clouds are sparse, unordered, and self-occluded, and with 0.3–15% of scene points moving, the slow-feature optimum of latent prediction compounds with the geometric shortcut of 3D self-supervision. We lift three canonical JEPA designs to point clouds, frozen-encoder, distribution-prior, and action-sensitive, and re-sense the `stable-worldmodel` benchmark so that only the observation differs from the image baselines. All three plan without collapse: the distribution-prior model is statistically equivalent to its re-evaluated image counterpart on every benchmark, and the action-sensitive model attains the strongest result in our controlled comparison where the most geometry moves. Probing explains why: object positions are almost perfectly linearly decodable and attention falls on the few moving points. Planning withstands heavy dropout never seen in training, though range noise defeats the thinnest scene. Geometry finally makes a commanded 3D target a natural goal interface: we construct the goal latent from the target and the current latent, at no cost in success rate, without a goal observation.

1 Introduction

A robot that can predict the consequences of its own actions need not be programmed for each task: control reduces to search. World models promise this for physical AI [11, 12, 18]: learn dynamics from raw experience, offline and reward-free, and plan new behaviors at test time. Joint-embedding predictive architectures (JEPAs) made this practical by predicting future *latent* states conditioned on actions rather than generating observations, whether over frozen features (DINO-WM [48]), distribution-prior (LeWorldModel [21]), or action-sensitive (Delta-JEPA [46], SMWM [16]). Yet almost all of it sees the world through a camera.

Images are a rational default, with cheap sensors, dense semantics, and transferable foundation models [32]. But many robots perceive the world geometrically, and range perception has matured to match, recovering 3D curves [26] and 6-DoF object poses [33] from raw scans. What is missing is *prediction*, a world model over geometric observations that a planner can interrogate, and it is not obvious that latent prediction survives the move. Latent prediction admits a trivial optimum at temporally constant features [34], unusually accessible when only 1–15% of scene points move [15]. Independently, 3D self-supervision admits a geometric shortcut onto point height and surface normals, since coordinates enter the operators directly and cannot be masked [40]. A point-cloud

*Joint first authors: F.F.O. led algorithm development, M.S. led data generation and environment design.

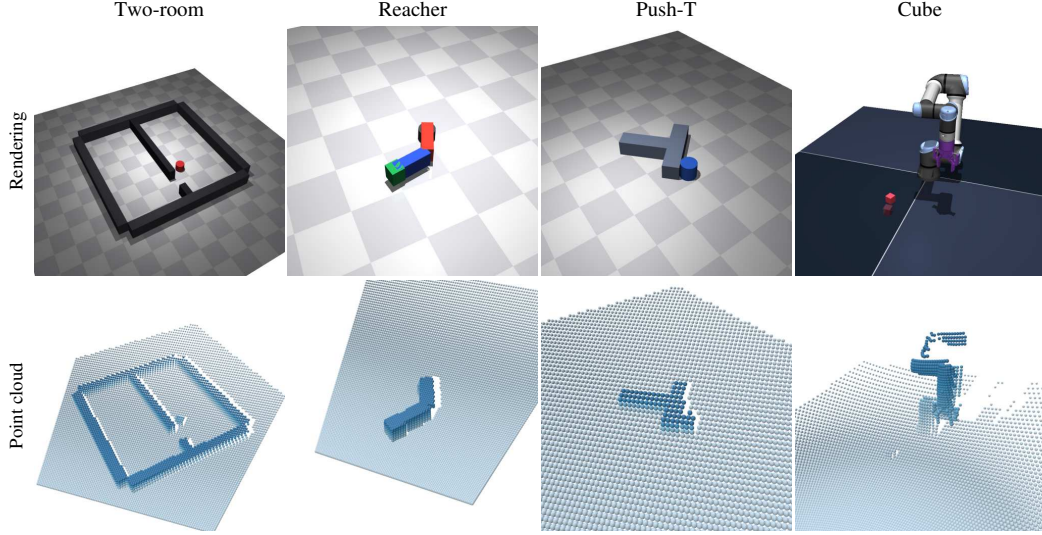


Figure 1: **From image environments to point cloud environments.** Top: the 3D scene. Bottom: the generated point cloud of that same scene, which replaces the image as the observation.

JEPA thus faces two routes to collapse where an image JEPA faces one, and whether the known anti-collapse mechanisms hold is an empirical question.

Existing 3D world models do not answer it: driving models generate future scans or occupancy judged by geometric fidelity [44, 47, 41, 19], manipulation models decode explicit future geometry [15, 29], and AD-L-JEPA [51] predicts latents but is action-free. To our knowledge, no prior work learns an action-conditioned latent-prediction world model on point clouds and plans with it.

We build that missing piece. We lift the three canonical JEPA designs to point clouds, isolating each known anti-collapse mechanism, and evaluate them against their image counterparts by re-sensing the `stable-worldmodel` suite [20] with a simulated range sensor.

Finally, latent planners require an encoded *goal observation* at inference, for a range sensor a scan of a configuration never visited. Geometry affords a natural interface: we learn to construct the goal latent from the current latent and a commanded 3D target, so the planner is told where an object should end up rather than shown a picture of the world once it is there. Our contributions:

- **Point-cloud JEPA world models:** frozen-encoder (Utonia-WM, a novel variant over the frozen Utonia encoder [45]), distribution-prior (Point-LeWM), and action-sensitive (Point-Delta-JEPA), plus Vox-WM, a parameter-free geometric ablation that Utonia-WM outperforms. On the sparse, near-static scans none collapses: positions stay linearly readable from the latents, attention settles on the moving entities, and planning largely withstands sensor degradation never seen in training.
- **A controlled modality comparison:** a re-sensed `stable-worldmodel` benchmark where only the observation differs from the image datasets, verified by replay. On it, Point-LeWM is statistically equivalent to the image baseline on every benchmark, and Point-Delta-JEPA exceeds it where the most geometry moves (above even its image counterpart’s reported, uncontrolled numbers).
- **Planning toward 3D targets:** a module predicting the goal latent from the current latent and a target object pose, replacing the inference-time goal observation without losing success.

2 Related Work

JEPA world models. Where classical world models train by reconstruction [11, 12] or reward prediction [13], JEPAs [18] train an action-conditioned predictor in representation space: DINO-WM [48] established the frozen-encoder variant, PLDM [35] demonstrated end-to-end latent planning,

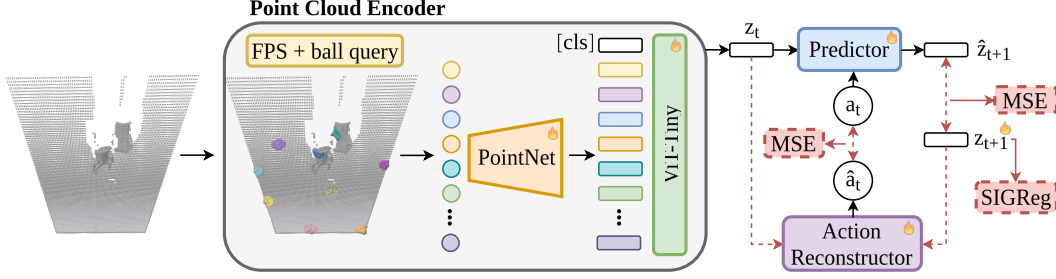


Figure 2: **Encoder of Point-LeWM and Point-Delta-JEPA:** FPS + ball query (only a few groups are shown), PointNet [30] tokenizer, and ViT-Tiny [8] whose CLS token yields z_t . The next latent z_{t+1} , from the same encoder, is matched to $\hat{z}_{t+1}$ via MSE. SIGReg [21] (Point-LeWM) and the Action-Reconstruction-Loss [16, 46] (Point-Delta-JEPA) prevent collapse.

and LeWorldModel [21] reduced end-to-end training to latent prediction plus one distributional regularizer (SIGReg [5]). Later work adds temporal-difference targets [4], test-time adaptation [37], action-sensitivity [10], and video-model post-training [3]. The `stable-worldmodel` platform [20] consolidates these models and planners into the interface we re-sense. Two failure modes organize this space. Against the slow-feature optimum [34], remedies range from distributional regularization [5, 43] and value heads [13] to an inverse-dynamics loss, that SMWM [16] and Delta-JEPA [46] parameterize on the latent displacement. Against one-step training degrading under multi-step rollouts, remedies include latent overshooting, multi-step targets, and exposure to the model’s own predictions [12, 3, 4, 24]. All are measured on images or proprioceptive states.

World models on point clouds. The existing lanes never plan in latent space. *Manipulation* models regress future geometry: PointWorld [15] predicts per-point displacements and drives MPPI on a real arm, 3D Point World Models [29] place point completion before the dynamics, and both pay their planning cost on a decoded cloud. *Driving* models are generative: Copilot4D [44] and OccWorld [47] synthesize future scans or occupancy, ViDAR [41] forecasts point clouds as pretraining, and LiSTAR [19] shows how strongly sensor geometry shapes architecture, but none searches over candidate actions. Closest to us, AD-L-JEPA [51] and its temporal extension [50] bring masked *latent* prediction to automotive scans, the latter validating SIGReg on 3D latents, yet both are evaluated by fine-tuning and neither conditions on actions. FR3D [24] predicts in a frozen 3D reconstruction latent space but is monocular and action-free. The frozen-encoder recipe also presupposes a general-purpose 3D encoder: Sonata [40] identifies the geometric shortcut of Section 1 in 3D self-supervision, and Utonia [45] trains a 3D foundation model across point-cloud domains.

Goal specification and planning. Latent world models plan by sampling-based MPC [12, 13, 48], and in the reward-free setting the goal enters as an *encoded goal observation* [20]. This interface degrades sharply with goal horizon [22, 6], and it presumes an observation of the goal state exists at all. FF-JEPA [22] removes goal images by forecasting latent subgoals, but its objective is implicit in the demonstrations, so the planner cannot be commanded to an arbitrary target. PRISM [38] and SAGE [6] keep the encoded goal and improve the search, and Nguyen et al. [25] amortize that search into a goal- and horizon-conditioned inverse-dynamics model. In 3D, PointWorld [15] plans toward target positions for a selected subset of points, in effect a sparse goal cloud with known correspondences, a cost only a model predicting explicit geometry can evaluate. Outside world models, coordinate goals are standard, from goal-conditioned RL [2] to PointGoal navigation [1]. We investigate another way: retain an explicit goal specification, represent it as the 3D pose of a task-relevant object, and learn to construct the goal latent the planners above lack without a goal observation.

3 Methods

Section 3.1 equips the `stable-worldmodel` environments with simulated LiDAR. Section 3.2 instantiates the three canonical JEPA world-model designs on the resulting point clouds and presents a complementary module that removes the need for a goal observation at inference time.

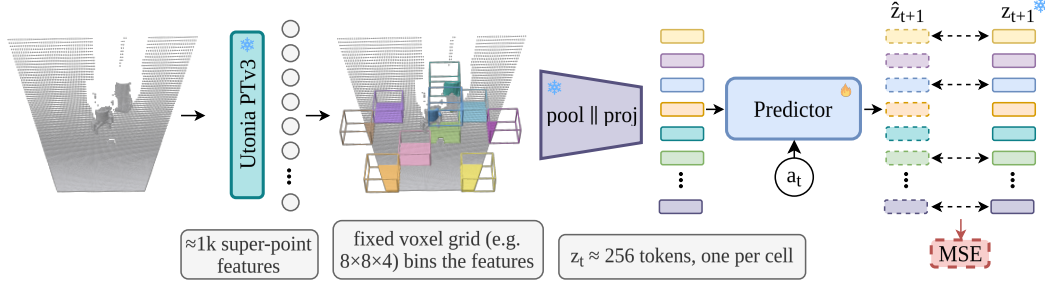


Figure 3: **Utonia-WM**: frozen Utonia [39, 45] encodes the point cloud into super-point features, which a fixed canonical-frame voxel grid bins into ≈ 256 tokens (only a few cells shown). Per cell, features are pooled and compressed by a fixed random orthogonal projection to the predictor width. Only the predictor trains, with a token-wise MSE against the frozen targets z_{t+1} .

Table 1: The four re-sensed datasets. *Ground share* and *moving returns* are the fractions of returns on the ground plane and on non-static entities. Both are constant within an environment.

	OGB-Cube	Two-Room	Push-T	Reacher3D
Sensor pose	scene camera	derived oblique	derived oblique	derived oblique
Ground share	$\sim 79\%$	$\sim 80\%$	$\sim 98\%$	$\sim 97\%$
Moving returns	$\sim 15.0\%$	$\sim 0.3\%$	$\sim 0.5\%$	$\sim 2.0\%$
Episodes	10,000	10,000	18,685	10,000
Frames	2,010,000	920,809	2,336,736	2,010,000
Episode length	201	31–101	49–246	201

3.1 Point-cloud sensing and dataset replication

To compare observation modalities rather than datasets we re-sense the released image benchmarks instead of collecting new data: a raycast range sensor is attached to the simulator behind each original dataset, every recorded state is replayed, and the resulting cloud is stored row-aligned with the original observation. Episodes, lengths, actions and goals are identical to the image datasets. Table 1 states per environment what else changes.

Sensor. The sensor casts a fixed 100×100 grid of rays matched to its mount camera’s field of view, recording each ray’s first intersection within a per-environment maximum range. Returns are expressed in the sensor frame (x forward, y left, z up), and rays intersecting nothing take a sentinel value, so each frame is a fixed $(10,000, 3)$ array. The sensor is idealized, exact intersections, no noise, dropout or sweep distortion, so the comparison isolates observation geometry. Only the OGBench cube [27] supplies a mountable camera. The others offer a top-down view alone, from which a scan is degenerate, so we place the sensor obliquely at 45° azimuth and elevation, derived from the scene’s bounding sphere rather than tuned, a harder view than the top-down render the image baselines see.

Dataset replication. Conversion writes each recorded state onto the environment and casts a scan without stepping physics. The output table carries every source column alongside the cloud. Two-Room and Push-T [7] are not 3D simulators, so we leave each engine unmodified and mirror its state into a lightweight MuJoCo [36] scene serving only as geometry for the sensor (Appendix C). Dynamics, rewards and termination remain the original environment’s. Ground returns dominate every cloud and only 0.3–15% fall on moving entities (Table 1), the regime Section 1 identifies as doubly hazardous for latent prediction. At evaluation the same sensor is cast from each state the planner visits.

3.2 Point cloud world models and planning toward 3D targets

We adopt the offline, reward-free setting of prior JEPA world models [16, 21, 48]: the training data is a set of n action-interleaved trajectories $\mathcal{D} = \{(o_1, a_1, \dots, a_{T_i-1}, o_{T_i})\}_{i=1}^n$, without rewards or task labels. Each observation o_t is a point cloud, an unordered point set in $\mathbb{R}^3$ of frame-varying size, and each action a_t a continuous control input. At test time the model must plan actions that reach a

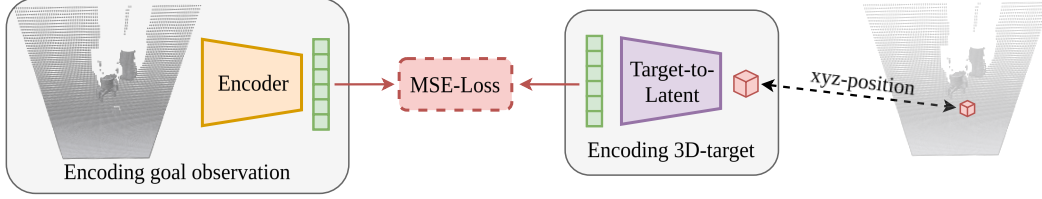


Figure 4: **Learning to plan toward 3D targets.** The target-to-latent module learns to reproduce the frozen encoder’s latent of a stored goal observation (left) from a task-relevant 3D target, in the cube environment shown here simply the xyz -position of the cube. At test time the 3D target is supplied directly, replacing the need for a goal observation.

specified goal, conventionally given as a *goal observation* o_g [20, 48]. Our target-to-latent module will instead assume training-time annotations c_t of low-dimensional 3D targets, the only supervision used in this paper.

3.2.1 Model architecture

All models share the JEPA template [18]: an encoder f_θ maps each observation to a latent $z_t = f_\theta(o_t) \in \mathcal{Z}$, and a predictor p_ϕ predicts the next latent from a context of ℓ latents and actions, $\hat{z}_{t+1} = p_\phi(z_{t-\ell+1:t}, a_{t-\ell+1:t})$. Observations are never reconstructed. Point-LeWM and Point-Delta-JEPA are trained end-to-end with an *identical* encoder and predictor and differ only in the objective, which isolates the anti-collapse mechanism. Utonia-WM trains no encoder at all.

Point-LeWM & Point-Delta-JEPA. For the end-to-end pair, f_θ is the point-cloud analogue of LeWorldModel’s pixel encoder (Figure 2): a Point-BERT-style set tokenizer [42, 49] feeds the image model’s ViT-Tiny trunk [8], whose CLS readout gives $\mathcal{Z} = \mathbb{R}^{192}$. The predictor is LeWorldModel’s causal action-conditioned transformer [21]. Three choices depart from recognition practice. Coordinates are normalized by a *fixed* workspace affine rather than per cloud, since per-cloud statistics would erase the absolute object positions that constitute the task signal. Points are grouped by metric radius [31] rather than by k -NN, so every token covers a fixed physical scale. Finally, the encoder uses LayerNorm exclusively, R3D’s fix [14] for the BatchNorm-induced 3D “scaling paradox” in control.

Utonia-WM & Vox-WM. Utonia-WM (Figure 3) follows DINO-WM [48]: a frozen pretrained encoder supplies the state, and only the predictor trains. Our frozen encoder is Utonia [45], a Point Transformer V3 [39] consuming point coordinates alone. Unlike an image backbone’s patch tokens, Utonia’s super-points carry no persistent identity across frames, so DINO-WM’s token-wise prediction is undefined, and we impose the missing structure deterministically: each point cloud is expressed in a fixed gravity-aligned frame, encoded by the frozen backbone, and pooled into a fixed grid of canonical-frame cells, ≈ 256 tokens (Appendix A). Each pooled token is then compressed from its 1152 pooled dimensions to the 192-dimensional predictor width by a fixed random orthogonal projection (Appendix A), training-free and approximately distance-preserving in the sense of the Johnson–Lindenstrauss lemma [17]. These tokens are predicted by DINO-WM’s frame-block-causal transformer [48]. Nothing downstream of the backbone is trainable, so the frozen latent space cannot collapse under the prediction loss. Vox-WM ablates the frozen encoder at fixed tokenization: it keeps the canonical frame, the token grid, and the predictor, but each cell’s token holds 25 plain statistics of the cell’s points (occupancy, count, and per-axis moments, Appendix A) instead of pooled frozen features, so whatever Utonia-WM gains over Vox-WM is attributable to the learned features rather than to the voxel representation itself.

Joint training objectives. All models minimize the teacher-forced latent prediction loss $\mathcal{L}_{\text{pred}} = \mathbb{E}_{\mathcal{D}} [\|p_\phi(z_{t-\ell+1:t}, a_{t-\ell+1:t}) - z_{t+1}\|_2^2]$ [48, 21], i.e. the predicted next latent must match the encoded next observation. Trained end-to-end, this loss alone is degenerate, since a constant encoder attains zero loss [34]. Our three models occupy the three known escape routes. Point-LeWM optimizes $\mathcal{L}_{\text{pred}} + \lambda \Omega_{\text{SIGReg}}(\theta)$, where the regularizer SIGReg [5, 21], weighted by λ , drives the latent distribution toward an isotropic Gaussian, which a collapsed encoder cannot produce. Point-Delta-

JEPA instead optimizes $\mathcal{L}_{\text{pred}} + \lambda \mathbb{E}_{\mathcal{D}} \left[\frac{1}{N} \|h_{\psi}(z_{t+N} - z_t) - a_{t:t+N-1}\|_2^2 \right]$, in which a training-only decoder h_{ψ} , discarded after training, must recover the $N=5$ executed actions from the net latent displacement, forcing latents that discriminate between actions [46, 16]. Utonia-WM needs neither term, since only the predictor is trained, against the fixed targets of the frozen encoder [48], at the price that the representation cannot adapt to the control task.

Implementation. All models are implemented in PyTorch [28] on a deliberately modest budget: every world model trains on two A100 GPUs. The shared end-to-end encoder is a ViT-Tiny trunk of $\approx 5.5\text{M}$ parameters with a 192-dimensional latent, trained for 10 epochs, whereas Utonia-WM pairs its frozen $\approx 137\text{M}$ -parameter backbone with a predictor trained for 100 epochs on precomputed features. All models step at frameskip 5 and plan from a context of $\ell = 3$ past latents. Architecture and optimization constants otherwise follow the image pipelines [21, 48] (Appendix A).

3.2.2 Planning toward 3D targets

Planning follows the pixel models [21, 48]: receding-horizon MPC in latent space, minimizing the distance $\|\hat{z}_{t+H} - z_g\|_2^2$ to a goal latent z_g over action sequences of horizon H with the cross-entropy method, where $\hat{z}_{t+H}$ is the autoregressive rollout of p_{ϕ} . Planner and CEM hyperparameters are inherited unchanged from the image pipelines (Appendix A), so the comparison never rests on planner tuning. Conventionally, the goal latent is an encoded goal observation, $z_g = f_{\theta}(o_g)$, for LiDAR a complete scan of a configuration the robot has never visited. What is actually available in practice [33] is a low-dimensional 3D target $c_g \in \mathcal{C}$, instantiated per environment as the cube position, the agent position (Two-Room), or agent and block positions plus the block’s planar angle (Push-T). A target alone need not index a latent: f_{θ} encodes the entire scene while c_g constrains only the task-relevant coordinates, so the map from target to latent is one-to-many. Conditioning on the *current* latent z_t , which supplies exactly the scene content the target leaves unconstrained, resolves this ambiguity. Without it, the regression target of Eq. (1) averages over every scene consistent with the target and need not be the latent of *any* reachable observation (Appendix B). We evaluate both variants in Table 3. With f_{θ} frozen, we train $m_{\xi} : \mathcal{C} \times \mathcal{Z} \rightarrow \mathcal{Z}$ on pairs of frames from the same trajectory, separated by a random offset $k \leq k_{\text{max}}$:

$$\mathcal{L}_{\text{goal}}(\xi) = \mathbb{E}_{(o_t, c_{t+k}, o_{t+k}) \sim \mathcal{D}, k \sim \mathcal{U}\{1, \dots, k_{\text{max}}\}} \left[\|m_{\xi}(c_{t+k}, z_t) - z_{t+k}\|_2^2 \right]. \quad (1)$$

We instantiate m_{ξ} either as a residual MLP over Fourier-featurized target coordinates, which fits the conditional mean of Eq. (1), or as a shortcut model [9], a generative alternative that samples z_g from the full distribution of latents consistent with a target, in a single step at the MLP’s inference cost (architectures and training constants in Appendix A). The unconditioned variant drops z_t from both the module and the loss. At test time $z_g = m_{\xi}(c_g, z_t)$ or $z_g = m_{\xi}(c_g)$ replace the encoded goal observation in the planning objective. World model and planner are untouched, so the module is model- and modality-agnostic. Appendix B discusses the alternative of casting a synthetic goal scan.

4 Experiments

All models are trained on the re-sensed datasets of Section 3.1 and evaluated by goal-conditioned planning behind the `stable-worldmodel` solver layer [20]: receding-horizon CEM over 10 seeds of 50 episodes each, on a fixed list of start–goal pairs shared across all models and image baselines.

4.1 Baselines

Table 2 answers the title question in the affirmative: every point-cloud world model plans far above Random in every environment, despite moving fractions of only 0.3–15% (Table 1), so neither route to degeneracy of Section 1 is realized. Against image-space LeWM, re-evaluated from its released checkpoints [21] on identical start–goal pairs, Point-LeWM is statistically equivalent on every benchmark, with only a small but significant 2.8-point deficit on Push-T (tests in Appendix E). On Two-Room it even attains the higher mean. Sparsity, missing ordering and self-occlusion thus cost at most a few points of planning performance, and little compute: 0.53 s per CEM solve versus 0.48 s (Appendix A).

Objective and modality interact. Point-Delta-JEPA saturates Two-Room and attains the best OGB-Cube result in our controlled evaluation (83.4 ± 3.8 , above even its image-space counterpart [46]),

Table 2: Planning success rate (%), mean $\pm$ sd over 10 seeds $\times$ 50 episodes (sd across seeds, here and throughout), with CEM behind the `stable-worldmodel` solver [20] on an identical fixed list of start-goal pairs. Rows marked $\dagger$ are copied from prior work under a different protocol (DINO-WM with 1 seed and no sd [21], Delta-JEPA with 3 seeds [46]) and serve as context only.

Family	Obs.	Method	Two-Room	Reacher	Push-T	OGB-Cube
		Random	25.2 $\pm$ 4.1	10.8 $\pm$ 4.0	2.4 $\pm$ 1.3	46.0 $\pm$ 8.5
SIGReg	Image Points	LeWM [21]	85.6 $\pm$ 5.7	82.0 $\pm$ 5.6	86.4 $\pm$ 5.2	69.8 $\pm$ 9.3
		Point-LeWM	87.0 $\pm$ 4.4	80.4 $\pm$ 4.6	83.6 $\pm$ 3.4	66.0 $\pm$ 5.9
Act.-recon.	Image Points	<i>Delta-JEPA</i> † [46]	100.0 $\pm$ 0.0	81.3 $\pm$ 0.5	89.1 $\pm$ 1.9	79.3 $\pm$ 1.8
		Point-Delta-JEPA	100.0 $\pm$ 0.0	77.6 $\pm$ 4.4	70.8 $\pm$ 7.7	83.4 $\pm$ 3.8
Frozen enc.	Image Points	<i>DINO-WM</i> † [48]	100.0	79.0	74.0	86.0
	Points	Utonia-WM	94.0 $\pm$ 1.9	71.2 $\pm$ 3.7	46.6 $\pm$ 6.6	71.2 $\pm$ 7.7
	Points	Vox-WM	81.4 $\pm$ 4.8	54.4 $\pm$ 6.2	35.2 $\pm$ 6.3	63.4 $\pm$ 7.8

significantly outperforming Point-LeWM on both (+13.0 and +17.4 points, $p < 0.001$). Push-T reverses the ranking (70.8 vs. 83.6, $p < 0.001$). The probing below offers a mechanism: the task scores the block’s orientation, which Point-Delta-JEPA encodes less linearly (Table 4). On Reacher the two objectives are statistically indistinguishable (77.6 vs. 80.4, $p = 0.30$). No single objective dominates, so the anti-collapse objective is a first-order, task-dependent design choice.

Utonia-WM shows the frozen-encoder recipe transfers to point clouds: it plans far above Random everywhere, is strongest on navigation (94.0 on Two-Room), and on OGB-Cube attains a higher mean than both SIGReg models (71.2 vs. 66.0 and 69.8, within noise). Manipulation on Push-T is its weak point (46.6), the price of a representation that cannot adapt to the control task. Its features are nonetheless useful rather than an artifact of the voxel representation: it beats its training-free counterpart Vox-WM on every benchmark (+7.8 to +16.8 points), Push-T included.

4.2 Ablation

The remaining analyses dissect Point-LeWM and Point-Delta-JEPA: what the latents encode, how planning survives a degraded sensor, and where the encoders look.

Physical latent probing. We regress privileged simulator state from the exact 192-dimensional latent the CEM cost uses, with a linear (ridge) and an MLP probe on episode-disjoint splits (Appendix D.1). Across all three environments, *positions* are decodable almost perfectly by both models, linearly at $R^2 \geq 0.95$ and near-exactly with the MLP (Tables 4–6). *Angles* are where the objectives separate: on Push-T (Table 4) the block angle is linearly decodable from Point-LeWM at $R^2 = 0.89$ but from Point-Delta-JEPA only at 0.66, while the MLP recovers it from both, so the information is present but not linearly organized. Near-perfect position probing does not imply planning success: on Push-T both models decode positions equally well, yet Point-LeWM plans 13 points better. The planning gap follows the angle probe, since the task scores the block’s orientation, which Point-LeWM encodes far more linearly. Conversely, on OGB-Cube only Point-Delta-JEPA decodes joint *velocity* at all (MLP R^2 0.63 vs. 0.10, Table 6), consistent with its action-reconstruction objective and with its stronger planning there.

Point cloud perturbations. We degrade the scan at evaluation time only, with along-ray range noise of $\sigma \in \{0.25, 0.5\}\%$ of the per-frame scan extent and dropout turning 25 or 50% of returns into misses, applied to the live observation alone or to observation and goal cloud alike. Table 7 (Appendix D.2) reports success relative to each model’s clean performance. Degrading the goal cloud as well costs a further, roughly uniform ~ 6 relative points, so the numbers below keep the goal clean. Point-LeWM degrades more gracefully than Point-Delta-JEPA in nearly every condition: under 0.25% range noise it retains 80% of its clean Two-Room performance where Point-Delta-JEPA keeps 43%, and it retains 99% and 95% under 50% dropout on Two-Room and Reacher where Point-Delta-JEPA drops to 64% and 87%, so the distribution-matching objective appears to yield the smoother, more redundant latent. Push-T is the shared failure case: noise collapses both models to

Table 3: Planning success rate (%) toward 3D targets, mean $\pm$ std over 10 seeds (50 episodes each). Target encoders (P-LeWM/PD-JEPA abbreviate Point-LeWM/Point-Delta-JEPA) map the 3D target to a goal latent. z_t marks whether the current latent is an additional input. *Goal cloud* encodes the stored goal cloud (reference). Best per column in bold, separately with and without z_t .

Target Encoder	z_t	Push-T		OGB-Cube		Two-Room	
		P-LeWM	PD-JEPA	P-LeWM	PD-JEPA	P-LeWM	PD-JEPA
MLP	$\times$	85.0 $\pm$ 3.6	71.8 $\pm$ 7.1	65.2 $\pm$ 6.5	72.0 $\pm$ 5.2	87.4 $\pm$ 4.8	100.0 $\pm$ 0.0
Shortcut	$\times$	83.2 $\pm$ 4.4	71.4 $\pm$ 6.8	61.6 $\pm$ 5.7	78.8 $\pm$ 5.3	87.6 $\pm$ 4.8	100.0 $\pm$ 0.0
MLP	$\checkmark$	83.8 $\pm$ 5.0	70.8 $\pm$ 7.4	66.0 $\pm$ 7.8	82.4 $\pm$ 5.9	88.4 $\pm$ 4.3	100.0 $\pm$ 0.0
Shortcut	$\checkmark$	84.8 $\pm$ 2.3	73.6 $\pm$ 5.8	68.8 $\pm$ 8.3	83.2 $\pm$ 5.0	88.6 $\pm$ 3.9	100.0 $\pm$ 0.0
<i>Goal cloud</i> (reference)		84.8 $\pm$ 4.1	71.2 $\pm$ 5.0	65.4 $\pm$ 7.5	84.2 $\pm$ 4.2	87.2 $\pm$ 4.1	100.0 $\pm$ 0.0

below 5% relative. Its scan is 98% ground plane and its task entities are thin, so a noise scale set by the whole scene is comparable to their height and effectively erases them. OGB-Cube remains at clean level for both models (98–103% under noise, 99–102% under dropout). It is also where the most points move ($\sim 15\%$), suggesting latents anchored to the actuated geometry rather than to incidental static structure. The CLS-attention rollouts (Appendix D.3) agree: they concentrate on the moving entities although these carry as little as 0.3% of the returns, so neither objective settled into the slow-feature optimum.

4.3 Towards planning with 3D-targets

Table 3 evaluates the target-to-latent module of Section 3.2 against planning toward the encoded goal cloud, and shows the goal observation is dispensable: with the current latent as input, the learned encoders match the goal-cloud reference within seed noise in every column, so a planner can be commanded by a low-dimensional target pose at no cost. Conditioning on z_t matters where Section 3.2 predicts: on OGB-Cube, whose target leaves the arm configuration unconstrained, dropping z_t costs Point-Delta-JEPA up to 12 points, while on Push-T and Two-Room, whose targets constrain all moving entities, the unconditioned variants suffice. The generative shortcut is best or tied in every conditioned column, but its margin over the MLP lies within seed noise.

5 Conclusion

Limitations. Utonia-WM plans slowly: its predictor rolls out ≈ 256 latent tokens where the end-to-end models roll out one, a $\approx 50\times$ gap that mirrors the DINO-WM [21] and excludes it from the perturbation ablation. Its fixed grid pooling and random projection also impose token structure the end-to-end models learn, a plausible source of its manipulation gap, and the Vox-WM comparison carries a predictor-width confound (Appendix A). The target-to-latent module assumes training-time target annotations and is modality-agnostic (Appendix B). The study spans four simulated tabletop scenes with an idealized sensor, degraded only synthetically at test time (Appendix D.2). The $\dagger$ baselines remain uncontrolled: no image Delta-JEPA checkpoint is available, and the compute already spent precluded retraining it or re-evaluating DINO-WM.

Conclusion. Latent planning survives point clouds. We lifted the three canonical JEPA designs to geometric observations and showed that, despite sparse, self-occluded scans where as little as 0.3% of returns move, none collapses and all plan, to our knowledge the first point-cloud world models interrogated with candidate actions rather than judged by reconstruction. Re-sensing the stable-worldmodel benchmark holds episodes, actions, goals, and planners fixed. Under this controlled comparison the distribution-prior model is statistically equivalent to its image counterpart and the action-sensitive model exceeds it where the most geometry moves. Our analysis explains why: positions are almost perfectly linearly decodable, attention lands on the task-relevant entities, and planning withstands heavy unseen dropout, while range noise on thin structures remains the failure mode. Geometry then pays a practical dividend, a planner commanded by a 3D target pose instead of a goal observation at no cost. This work establishes range sensing as a first-class modality for world-model planning, and points toward the robot that must predict and act where cameras fail.

Acknowledgments

The computational results presented have been achieved using the Vienna Scientific Cluster (VSC).

References

- [1] Peter Anderson, Angel Chang, Devendra Singh Chaplot, Alexey Dosovitskiy, Saurabh Gupta, Vladlen Koltun, Jana Kosecka, Jitendra Malik, Roozbeh Mottaghi, Manolis Savva, and Amir R. Zamir. On evaluation of embodied navigation agents, 2018.
- [2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *NeurIPS*, 2017.
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Geji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning, 2025.
- [4] Marco Bagatella, Matteo Pirota, Ahmed Touati, Alessandro Lazaric, and Andrea Tirinzoni. TD-JEPA: Latent-predictive representations for zero-shot reinforcement learning, 2025.
- [5] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics, 2025.
- [6] Letian Cheng, Qi Zhang, and Yisen Wang. SAGE: Subgoal-conditioned action generation for latent world model planning, 2026.
- [7] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [9] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *International Conference on Learning Representations*, volume 2025, pages 34668–34684, 2025.
- [10] Zhenfeng Gan, ZiTong Zeng, Jiajun Cheng, Yeke Song, Yongyi Tang, and Xueqian Wang. ActSWM: Action-sensitive world models for long-horizon planning in open-world games, 2026.
- [11] David Ha and Jürgen Schmidhuber. World models, 2018.
- [12] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.
- [13] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *ICLR*, 2023.
- [14] Zhengdong Hong, Shenrui Wu, Haozhe Cui, Boyi Zhao, Ran Ji, Yiyang He, Hangxing Zhang, Zundong Ke, Jun Wang, Guofeng Zhang, and Jiayuan Gu. R3D: Revisiting 3D policy learning, 2026.
- [15] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation, 2026.

- [16] Petr Ivashkov, Randall Balestriero, and Bernhard Schölkopf. Sensorimotor world models: Perception for action via inverse dynamics, 2026.
- [17] William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- [18] Yann LeCun. A path towards autonomous machine intelligence, 2022. URL <https://openreview.net/forum?id=BZ5a1r-kVsf>.
- [19] Pei Liu, Songtao Wang, Lang Zhang, Xingyue Peng, Yuandong Lyu, Jiaxin Deng, Songxin Lu, Weiliang Ma, Xueyang Zhang, Yifei Zhan, XianPeng Lang, and Jun Ma. LiSTAR: Ray-centric world models for 4D LiDAR sequences in autonomous driving, 2025.
- [20] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.
- [21] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels, 2026.
- [22] Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPA: Long-horizon planning in world models with latent planners, 2026.
- [23] Ishan Misra, Rohit Girdhar, and Armand Joulin. An end-to-end transformer model for 3D object detection. In *ICCV*, 2021.
- [24] Nils Morbitzer, Jonathan Evers, Artem Savkin, Thomas Stauner, Nassir Navab, Federico Tombari, and Stefano Gasperini. Future dynamic 3D reconstruction: A 3D world model with disentangled ego-motion. In *ICML*, 2026.
- [25] Hoang Nguyen, Xiaohao Xu, and Xiaonan Huang. Latent geometry beyond search: Amortizing planning in world models, 2026.
- [26] Fabio F. Oberweger, Michael Schwingshackl, and Vanessa Staderini. PI3DETR: Parametric instance detection of 3D point cloud edges with a geometry-aware 3DETR. In *3DV*, 2026. doi: 10.1109/3DV69130.2026.00052.
- [27] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025.
- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- [29] Skand Peri, Hung Nguyen, Chanho Kim, Li Fuxin, and Stefan Lee. 3D point world models: Point completion enables more accurate dynamics learning, 2026.
- [30] Charles Ruizhongtai Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, 2017.
- [31] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *NeurIPS*, 2017.
- [32] Michael Schwingshackl, Fabio Francisco Oberweger, and Markus Murschitz. Few-shot structure-informed machinery part segmentation with foundation models and graph neural networks. In *WACV*, 2025. doi: 10.1109/WACV61041.2025.00200.
- [33] Michael Schwingshackl, Fabio F. Oberweger, Mario Niedermeyer, Johannes Huemer, and Markus Murschitz. PIRATR: Parametric object inference for robotic applications with transformers in 3D point clouds. In *ICRA*, 2026.

- [34] Vlad Sobal, Jyothir S, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. In *NeurIPS*, 2022.
- [35] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *NeurIPS*, 2025.
- [36] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [37] Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. AdaJEPa: An adaptive latent world model, 2026.
- [38] Yuhai Wang, Jiawei Xia, Rongxuan Zhou, Xiao Hu, Yongliang Shi, Jing Du, and Yang Ye. PRISM: PRior-guided imagination sampling in world models, 2026.
- [39] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer V3: Simpler, faster, stronger. In *CVPR*, 2023.
- [40] Xiaoyang Wu, Daniel DeTone, Duncan Frost, Tianwei Shen, Chris Xie, Nan Yang, Jakob Engel, Richard Newcombe, Hengshuang Zhao, and Julian Straub. Sonata: Self-supervised learning of reliable point representations. In *CVPR*, 2025.
- [41] Zetong Yang, Li Chen, Yanan Sun, and Hongyang Li. Visual point cloud forecasting enables scalable autonomous driving. In *CVPR*, 2024. doi: 10.1109/CVPR52733.2024.01390.
- [42] Xumin Yu, Lulu Tang, Yongming Rao, Tiejun Huang, Jie Zhou, and Jiwen Lu. Point-BERT: Pre-training 3D point cloud transformers with masked point modeling. In *CVPR*, 2022.
- [43] Zhoushun Yu, Xiaoyu Hu, and Xiangyu Xu. QQWorld: Quantile-quantile matching for world model regularization, 2026.
- [44] Lunjun Zhang, Yuwen Xiong, Ze Yang, Sergio Casas, Rui Hu, and Raquel Urtasun. Copilot4D: Learning unsupervised world models for autonomous driving via discrete diffusion. In *ICLR*, 2023.
- [45] Yujia Zhang, Xiaoyang Wu, Yunhan Yang, Xianzhe Fan, Han Li, Yuechen Zhang, Zehao Huang, Naiyan Wang, and Hengshuang Zhao. Utonia: Toward one encoder for all point clouds, 2026.
- [46] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPa: Learning action-sensitive world models via latent difference decoding, 2026.
- [47] Wenzhao Zheng, Weiliang Chen, Yuanhui Huang, Borui Zhang, Yueqi Duan, and Jiwen Lu. OccWorld: Learning a 3D occupancy world model for autonomous driving. In *ECCV*, 2024. doi: 10.1007/978-3-031-72624-8_4.
- [48] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *ICML*, 2025.
- [49] Junsheng Zhou, Jinsheng Wang, Baorui Ma, Yu-Shen Liu, Tiejun Huang, and Xinlong Wang. Uni3D: Exploring unified 3D representation at scale. In *ICLR*, 2024.
- [50] Haoran Zhu and Anna Choromanska. Self-supervised JEPa-based world models for LiDAR occupancy completion and forecasting, 2026.
- [51] Haoran Zhu, Zhenyuan Dong, Kristi Topollai, Beiyao Sha, and Anna Choromanska. Self-supervised representation learning with joint embedding predictive architecture for automotive LiDAR object detection. In *AAAI*, 2026. doi: 10.1609/AAAI.V40I16.38402.

A Implementation details

This section records the architecture and optimization constants behind Section 3.2. The released configurations remain authoritative. All world models are trained on two A100 GPUs.

Point-cloud encoder (Point-LeWM, Point-Delta-JEPA). The shared encoder (Figure 2) consumes geometry only: the per-point input features are the sensor-frame coordinates, with no intensity or color channel. Each cloud is reduced to 256 farthest-point-sampled centres, and a ball query of per-environment metric radius (0.15 m Two-Room, 0.18 m Push-T, 0.04 m Reacher) draws up to 32 points per centre uniformly at random [31]. Coordinates are first normalized by the fixed workspace map of Section 3.2, the affine $p \mapsto (p - c)/s$ with a per-environment centre $c \in \mathbb{R}^3$ and isotropic scale s measured once on the dataset, which brings the workspace to approximately $[-1, 1]^3$ while preserving absolute object positions. The centre-relative offsets of each group then pass through a two-stage mini-PointNet [30, 42] with hidden widths (128, 256), yielding one 192-dimensional token to which a log-spaced sinusoidal embedding of the group centre [23] is added. The tokens, prepended by a learned CLS token, feed a LayerNorm-only [14] ViT-Tiny trunk (12 layers, 3 heads, width 192, ≈ 5.5 M parameters) [8], whose CLS output is the latent z_t . OGB-Cube uses 512 tokens with radius 0.03 m instead. The denser sampling helped only there, since the other scenes are dominated by the ground plane.

Predictor, actions, and optimization. The predictor is LeWorldModel’s causal action-conditioned transformer [21]: 6 blocks, 16 heads of dimension 64, MLP width 2048 at the 192-dimensional latent width, with a context of $\ell = 3$ past latents. Latents pass a BatchNorm MLP projector (hidden 2048), as in the image model. The world models step at frameskip 5, the five intervening environment actions concatenated and embedded by a shared linear action encoder. Both end-to-end models train for 10 epochs with AdamW (learning rate 5×10^{-5} , weight decay 10^{-3}) at effective batch size 128, bf16 precision, and gradient clipping 1.0. The SIGReg weight is 0.09 (17 knots, 1024 projections) [5, 21]. The action-reconstruction weight is 10 at horizon $N=5$ [46], for which Point-Delta-JEPA trains on 6-frame windows while planning with the same 3-latent context. On OGB-Cube both models follow a 50-epoch cosine schedule, with the reported checkpoints still taken after 10 epochs.

Utonia-WM. Completing Section 3.2: point clouds are expressed in a canonical frame derived from the fitted ground plane, rescaled by Utonia’s own tabletop factor of 4.0, and voxel-downsampled at a grid size of 0.01, keeping one point per occupied voxel, before entering the frozen ≈ 137 M-parameter PTv3 backbone [39, 45]. Per grid cell, the backbone’s coarsest-stage 576-dimensional super-point features are mean- and max-pooled (1152 dimensions) and compressed to the 192-dimensional token width by the fixed random orthogonal projection of Section 3.2, a Gaussian matrix drawn once from a fixed seed and orthonormalized via QR. The grids are $8 \times 8 \times 4 = 256$ tokens (OGB-Cube, Reacher), $12 \times 11 \times 2 = 264$ (Push-T), and $10 \times 9 \times 3 = 270$ (Two-Room), spanning a per-environment workspace box. The predictor is DINO-WM’s frame-block-causal token transformer [48] at the same depth and widths as above, with the action embedding tiled over a frame’s tokens and concatenated onto each. Only the predictor and action encoder train, against precomputed frozen features, for 100 epochs with AdamW (learning rate 10^{-3} , weight decay 10^{-3}) at effective batch size 512.

Vox-WM. Vox-WM is a parameter-free geometric control for Utonia-WM that ablates the frozen foundation encoder at fixed tokenization: same canonical frame, same token grid, same predictor and prediction-only loss, but each cell’s token holds plain statistics of the cell’s points instead of pooled frozen features. After the voxel downsampling above, a cell’s points are expressed as offsets from the cell centre, and the 25-dimensional token stacks an occupancy bit (1), the point count as $\log(1+n)/\log(1+256)$ (1), the per-axis mean (3), standard deviation (3), minimum (3), and maximum (3) of the offsets, their three off-diagonal covariances (3), and the occupancy bits of a $2 \times 2 \times 2$ subdivision of the cell (8). Empty cells are zero tokens, which the occupancy bit disambiguates. The encoding is deterministic and costs microseconds per cloud, so it runs live without a feature cache, and the predictor is identical but runs at the native 25-dimensional token width. Training follows the Utonia-WM protocol. The native width leaves a capacity confound: Utonia-WM’s predictor operates at width 192, so part of its advantage over Vox-WM in Table 2 may reflect the wider predictor rather than the frozen features. A matched control projecting the 25 statistics to width 192, mirroring the random projection above, was not run.

Planning. Planner and CEM hyperparameters are inherited unchanged from the image pipelines [21, 16, 46], which share one configuration behind the `stable-worldmodel` solver layer [20].

Target-to-latent module. Both instantiations of m_ξ (Section 3.2) share one conditioning interface. The target coordinates, namely the cube position (3-d), the Two-Room agent position (3-d), or the Push-T agent and block positions together with the block’s heading vector (9-d), are expressed in the sensor frame, normalized by the same workspace map $p \mapsto (p - c)/s$ as the encoder inputs, and lifted by log-spaced sinusoidal Fourier features (32 bands) [23], which make sub-centimetre offsets linearly separable. The cube target carries no orientation channel, for the symmetry reason of Appendix D.1. The *MLP* head is a residual MLP (width 1024, hidden 2048, 4 blocks) regressing the normalized goal latent. The heading channels and, when used, z_t enter through a linear embedding added to the Fourier code. The *shortcut* head [9] is an adaptive-LayerNorm-modulated residual MLP of the same widths over the normalized latent space, trained with flow matching plus the self-consistency loss against an EMA copy (decay 0.999) and sampled in a single step at evaluation. Here the heading channels and z_t join the conditioning vector that modulates every block. Training pairs are frames of one trajectory separated by ± 5 to ± 25 environment steps, with z_t the latent of the earlier frame and the target read from the later. At evaluation z_t is re-encoded at every replan. All heads train for 60 epochs with AdamW (learning rate 3×10^{-4} , weight decay 10^{-4}) at batch size 4096 on precomputed frozen latents, with 5 mm Gaussian jitter on the position channels, without which the Fourier code is sharp enough to memorize individual frames.

Compute. Every world model trains on two A100 GPUs of an internal cluster. One end-to-end training run (Point-LeWM or Point-Delta-JEPA) takes roughly two to three days, and the Utonia-WM predictor trains in comparable time on precomputed frozen features. One evaluation sweep of 10 seeds $\times$ 50 episodes costs a few GPU-hours per model and environment, considerably more for Utonia-WM (Section 5). Completing the planning-cost numbers of Section 4: one CEM solve (300 candidates, 30 iterations, horizon 25) takes 0.53 s for the point-cloud model and 0.48 s for the image model on a single A100, and with replanning every 25 steps a 50-step episode requires about 1.1 s and 1.0 s of total planning time, an overhead shared by Point-Delta-JEPA whose encoder is identical. Including preliminary and failed runs, the full project consumed on the order of several thousand A100-hours.

Licenses. All existing assets are used under their published licenses: the `stable-worldmodel` suite (0.1.1) and its released LeWM checkpoints [20] (MIT), OGBench 1.2.1 [27] (MIT), the Push-T environment [7] (MIT), MuJoCo 3.10.0 [36] (Apache 2.0), the pretrained Utonia backbone [45] (Apache 2.0), and PyTorch 2.7.0 [28] (BSD-3-Clause).

B The unconditioned target-to-latent objective

This section makes precise the claim of Section 3.2 that, without the current latent, the regression target of Eq. (1) need not be the latent of any reachable observation. Write $\mathcal{Z}_f = \{f_\theta(o) : o \in \text{supp } \mathcal{D}\}$ for the set of latents the frozen encoder assigns to observations in the support of the data distribution $\mathcal{D}$, that is, to observations that actually occur in the data, and let (z_t, c_{t+k}, z_{t+k}) be distributed as in Eq. (1). The unconditioned variant minimizes $\mathbb{E}[\|m_\xi(c_{t+k}) - z_{t+k}\|_2^2]$, and over unconstrained functions $m : \mathcal{C} \rightarrow \mathcal{Z}$ this mean-squared error is minimized pointwise by the conditional mean

$$m^*(c) = \mathbb{E}[z_{t+k} \mid c_{t+k} = c], \quad (2)$$

the barycenter of the latents of *all* data scenes whose annotation equals c , covering every configuration of the scene content the target leaves free, on OGB-Cube every arm pose. $\mathcal{Z}_f$ is the image of the low-dimensional scene configuration manifold under the continuous encoder f_θ , hence itself a low-dimensional subset of $\mathbb{R}^{192}$, and no term of the training objective encourages it to be convex. SIGReg in particular shapes one-dimensional projections of the latent distribution rather than the geometry of its support. A barycenter of distinct points of a non-convex set need not lie in the set, and $m^*(c)$ can therefore be a latent that no observation in the data distribution produces. A planner minimizing $\|\hat{z}_{t+H} - m^*(c)\|_2^2$ is then steered toward a goal latent it may be unable to attain.

Conditioning changes the average, not the argument: with z_t as input the minimizer becomes $\mathbb{E}[z_{t+k} \mid c_{t+k} = c, z_t]$, an average over futures of the current scene only. Scene content that z_t determines and that the dynamics leave unchanged over the offset window contributes no variance to

this average, so the residual spread stems only from the moving entities the target leaves unconstrained. Whether that residue matters is an empirical question, and Table 3 answers it per environment: on Push-T and Two-Room the target constrains all moving entities and the unconditioned variant already matches the goal-cloud reference, while on OGB-Cube the arm remains free at the goal even given z_t , and dropping z_t costs up to 12 points. The two heads of Section 3.2 also treat the average differently: the MLP fits the conditional mean of Eq. (2) directly, while the shortcut model samples from the conditional distribution and so returns one consistent latent rather than a barycenter, although Table 3 separates the two heads only within seed noise.

Why not synthesize the goal scan? Since we control the mirror geometry, one could place the task-relevant entity at the commanded target, cast a synthetic scan of that state, and encode it, with no learned module. The construction founders on the same ambiguity as the unconditioned regression: a scene must be fully configured before a scan can be cast, the target constrains only the task-relevant coordinates, so the synthesizer has to invent the remaining content, on OGB-Cube an arm pose, and an invented configuration need not be reachable or consistent with the present scene. The conditioned module resolves this from z_t without reconstructing geometry, and in deployment the mismatch grows, since no perfect mirror of the scene exists to render from. The module itself is also modality-agnostic: given the same target annotations it could be trained on image latents. What geometry specifically contributes is that the target already lives in the metric space of the observation, so no cross-modal correspondence must be learned.

C Point-cloud sensing and dataset replication: details

Exact sensor parameters, mirror geometry and scene constants are given by the released configurations. This section records only the choices behind them.

Ray budget. The tokenizer reduces every cloud to 256 farthest-point-sampling centres, so the ray grid fixes the sampling density of the observation rather than the sequence length seen by the model. We use 100×100 as the coarsest grid at which the smallest task-relevant entity, the Push-T pusher or the Two-Room agent, still returns on the order of a hundred points.

Viewpoint. Oblique viewpoints are derived rather than hand-placed: given bounding-sphere radius R the camera sits at $d = 2.5R$ along the 45° azimuth and elevation bearing, with vertical field of view $2 \arcsin(R/d)$, so no pose is tuned per environment. The OGBench cube is the exception: its scene already contains the oblique camera its released images were rendered from, and the sensor is mounted on it, so that its scan and the image baseline’s observation share a viewpoint.

Mirror scenes. Entity heights in the two mirrored environments are ours to choose rather than inherited from the benchmark, and were fixed once per environment: large enough that an entity is separable from the floor in an oblique scan, small enough that self-occlusion does not dominate. Reacher3D is native, but the geometry of the released environment did not match the dataset it shipped with. We recovered the two link lengths by least-squares fitting to the recorded joint angles and fingertip positions, after which forward kinematics reproduces the recorded fingertip.

Goal handling. The goal marker is excluded from every scan, by an alpha threshold in the cube and by omission from the mirror geometry elsewhere, since a goal specifies a task rather than a surface a range sensor would return, and retaining it would permit a policy to read the goal off the observation rather than infer it. The goal reaches the planner as the stored cloud of the goal frame, indexed from the same converted table by the same goal offset the image pipeline uses, so no separate goal scan is cast.

D Ablations

D.1 Physical latent probing

Protocol. For each environment the frozen checkpoints of the two end-to-end world models encode every tenth frame of its training table. The probed representation is $z = f_\theta(o)$ after the projector, i.e.

Table 4: Physical latent probing results on Push-T. All targets are 3-dimensional: locations are coordinates in the sensor frame, and the block angle is encoded as a heading vector in the sensor frame. Lower MSE and higher R^2 indicate better representation quality.

Property	Method	Linear		MLP	
		MSE ↓	R^2 ↑	MSE ↓	R^2 ↑
Agent Location	Point-LeWM	0.008	0.955	0.001	0.997
	Point-Delta-JEPA	0.007	0.957	0.002	0.989
Block Location	Point-LeWM	0.004	0.958	0.000	0.998
	Point-Delta-JEPA	0.005	0.946	0.001	0.992
Block Angle	Point-LeWM	0.027	0.891	0.002	0.992
	Point-Delta-JEPA	0.084	0.657	0.009	0.962

Table 5: Physical latent probing results on Reacher. Positions are 3D coordinates in the sensor frame, and joint angles are encoded as per-link unit heading vectors in the sensor frame. Lower MSE and higher R^2 indicate better representation quality.

Property	Method	Linear		MLP	
		MSE ↓	R^2 ↑	MSE ↓	R^2 ↑
Finger Position	Point-LeWM	0.000	1.000	0.000	1.000
	Point-Delta-JEPA	0.000	0.999	0.000	1.000
Elbow Position	Point-LeWM	0.000	1.000	0.000	1.000
	Point-Delta-JEPA	0.000	0.999	0.000	1.000
Joint Angles	Point-LeWM	0.000	1.000	0.000	1.000
	Point-Delta-JEPA	0.001	0.998	0.000	1.000

exactly the 192-dimensional latent the CEM cost operates on. Two probes per target regress privileged simulator state from z : a *linear* probe, closed-form ridge regression on standardized features with the regularization weight selected on a validation split over a logarithmic grid, and an *MLP* probe ($192 \rightarrow 512 \rightarrow 512 \rightarrow d$, GELU, AdamW with cosine schedule, early-stopped on the same validation split). Splits are drawn at *episode* granularity (72/8/20 train/val/test), so every test frame comes from a trajectory no probe ever saw. We report test-set MSE in native units and pooled R^2 . All targets live in the sensor frame of the stored clouds, and planar angles enter as unit heading vectors, $(\cos a, \sin a, 0)$ rotated into the sensor frame, so that no probe has to fit a wrap-around discontinuity.

Results. Table 4 reports Push-T, analyzed in the main text. Tables 5 and 6 complement it on Reacher and OGB-Cube. Reacher is saturated: the arm is fully visible from the oblique viewpoint, and every property, positions and link headings alike, is linearly decodable at $R^2 \approx 1$ from both models, which is why it offers no signal for separating the objectives. On OGB-Cube the two yaw rows require care: a cube is 24-fold rotationally symmetric, so its orientation is not identifiable from geometry at all, and the raw yaw label is not folded into a fundamental domain. The near-zero R^2 therefore reflects an unidentifiable *label*, not a representation failure. The end-effector yaw is similarly flat for both models. The wrist subtends only a handful of returns at the 100×100 ray budget, and neither objective rewards retaining it. Joint velocity, finally, is not observable from a single scan and can be decoded only through its correlation with configuration along the data distribution. That Point-Delta-JEPA alone recovers a substantial share of it (R^2 0.63 vs. 0.10 with the MLP probe) is consistent with its objective, which forces the latent *displacement* to be decodable into the executed actions.

D.2 Point cloud perturbations

Protocol. Perturbations are applied at evaluation time only, downstream of a faithful scan. No model ever saw a degraded cloud in training. Test-time-only degradation is the probe matched to our question: it asks whether latents trained on clean geometry anchor to structure a worse sensor still

Table 6: Physical latent probing results on OGB-Cube. Positions are 3D coordinates in the sensor frame, stacked across the five arm joints for *Joint Positions*. Yaw is encoded as a heading vector in the sensor frame, and joint velocity is the raw six-dimensional vector in rad/s. Lower MSE and higher R^2 indicate better representation quality. The near-zero yaw rows reflect label unidentifiability (a cube is rotationally symmetric, and the wrist returns few points), not representation failure, as discussed in the text.

Property	Method	Linear		MLP	
		MSE ↓	R^2 ↑	MSE ↓	R^2 ↑
Joint Positions	Point-LeWM	0.000	0.992	0.000	0.999
	Point-Delta-JEPA	0.000	0.998	0.000	1.000
Joint Velocity	Point-LeWM	0.142	0.021	0.130	0.102
	Point-Delta-JEPA	0.071	0.507	0.054	0.626
End-Effector Position	Point-LeWM	0.000	0.992	0.000	0.999
	Point-Delta-JEPA	0.000	0.999	0.000	1.000
End-Effector Yaw	Point-LeWM	0.330	−0.001	0.332	−0.006
	Point-Delta-JEPA	0.321	0.029	0.321	0.026
Block Position	Point-LeWM	0.001	0.956	0.000	0.996
	Point-Delta-JEPA	0.000	0.996	0.000	0.999
Block Yaw	Point-LeWM	0.333	−0.001	0.335	−0.006
	Point-Delta-JEPA	0.333	−0.001	0.335	−0.006

measures, the situation of a model deployed on hardware noisier than its training data. Degrading the training data instead would measure whether the objectives can absorb noise, a property of training rather than of the learned representation. *Noise* is along-ray rather than isotropic, since a range sensor errs in the range it reports and each return therefore moves along its own line of sight. Its standard deviation σ is set as a fraction of the per-frame bounding-box diagonal of the valid returns. The relative scale is what makes the ablation comparable across environments: an absolute σ of 2 cm is a light haze on the 4.9 m Push-T arena and a catastrophe on the 1.4 m Reacher workspace. *Dropout* marks a random fraction of returns as misses, which is what a lost return physically is, so the dropped points leave through the same invalid-return filter the pipeline applies to genuine misses. Because the goal cloud is read from the dataset rather than scanned live, it stays clean under the first condition (a degraded sensor pursuing a stored target, the deployment case) and is degraded identically under the second, which tests the representation itself but no longer distinguishes “cannot see” from “aiming at the wrong thing”. Success rates in Table 7 are reported relative to each model’s own clean performance, so values may exceed 100 within seed noise.

Beyond the trends discussed in the main text, the table shows that degrading the goal in addition to the observation costs a further, roughly uniform margin (e.g. Two-Room noise 0.25%: 80.2 vs. 61.6 relative for Point-LeWM), and that dropout is benign wherever the surviving returns still sample the entities. Only Push-T, whose pusher is near the ray-budget floor of Appendix C, breaks under 50% dropout.

D.3 Attention

Figure 5 visualizes where the two end-to-end encoders look: the CLS-token attention is propagated through the ViT trunk by attention rollout, assigned from each group token to the points it covers, and normalised per cloud. Both models place their attention mass on the moving entities. In all four environments Point-LeWM is sharply peaked on the entities, while Point-Delta-JEPA allots visibly more attention to ground and table.

E Statistical tests

This section quantifies two claims of Section 4: that Point-LeWM plans on par with its image counterpart, and that the objective-family gaps are real. All tests operate on the ten per-seed success

Table 7: **Point cloud degradation.** Planning success rate over 10 seeds with 50 episodes each, reported *relative to each model’s own clean performance* (100 = no degradation). The top row gives the absolute clean success rates used as denominators. The two point-cloud world models are shown side by side under each condition, better of the pair in bold. Due to Utonia-WM’s long computation times, it is not part of this ablation table. Noise σ is a fraction of the frame’s scan extent, and dropout is the fraction of returns lost.

	Two-Room		Reacher		Push-T		OGB-Cube	
Perturbation	P-LeWM	PD-JEPA	P-LeWM	PD-JEPA	P-LeWM	PD-JEPA	P-LeWM	PD-JEPA
None	87.0 $\pm$ 4.4	100.0 $\pm$ 0.0	80.4 $\pm$ 4.6	77.6 $\pm$ 4.4	83.6 $\pm$ 3.4	70.8 $\pm$ 7.7	66.0 $\pm$ 5.9	83.4 $\pm$ 3.8
<i>Observation degraded, goal clean</i>								
Noise 0.25%	80.2 $\pm$ 9.4	42.8 $\pm$ 6.1	96.3 $\pm$ 8.2	89.4 $\pm$ 6.9	1.9 $\pm$ 1.9	3.7 $\pm$ 2.7	103.0 $\pm$ 10.8	99.8 $\pm$ 6.2
Noise 0.5%	52.2 $\pm$ 11.5	36.0 $\pm$ 6.7	90.8 $\pm$ 5.9	62.2 $\pm$ 6.1	1.2 $\pm$ 1.7	4.0 $\pm$ 2.0	97.6 $\pm$ 11.4	98.3 $\pm$ 6.8
Dropout 25%	99.8 $\pm$ 5.4	72.2 $\pm$ 6.4	97.5 $\pm$ 6.1	90.7 $\pm$ 6.5	90.7 $\pm$ 6.8	94.6 $\pm$ 8.8	101.5 $\pm$ 12.7	99.3 $\pm$ 4.8
Dropout 50%	98.9 $\pm$ 5.1	64.0 $\pm$ 6.7	94.8 $\pm$ 6.1	87.3 $\pm$ 5.6	49.0 $\pm$ 8.5	52.8 $\pm$ 10.5	99.4 $\pm$ 12.0	100.7 $\pm$ 4.9
<i>Observation and goal degraded</i>								
Noise 0.25%	61.6 $\pm$ 5.7	31.0 $\pm$ 5.1	91.8 $\pm$ 7.8	84.7 $\pm$ 6.0	3.6 $\pm$ 2.9	4.0 $\pm$ 3.2	97.0 $\pm$ 12.0	93.5 $\pm$ 6.8
Noise 0.5%	37.9 $\pm$ 6.0	35.8 $\pm$ 6.8	87.6 $\pm$ 7.5	58.0 $\pm$ 6.7	4.5 $\pm$ 2.4	2.5 $\pm$ 2.1	89.7 $\pm$ 15.9	77.2 $\pm$ 9.4
Dropout 25%	96.1 $\pm$ 5.9	60.4 $\pm$ 6.4	94.8 $\pm$ 5.0	89.1 $\pm$ 8.7	88.6 $\pm$ 6.3	87.0 $\pm$ 8.2	99.1 $\pm$ 12.3	98.8 $\pm$ 4.1
Dropout 50%	92.2 $\pm$ 4.6	50.2 $\pm$ 5.5	92.3 $\pm$ 5.8	75.9 $\pm$ 3.2	27.4 $\pm$ 5.5	35.3 $\pm$ 8.6	101.5 $\pm$ 8.5	99.5 $\pm$ 4.4

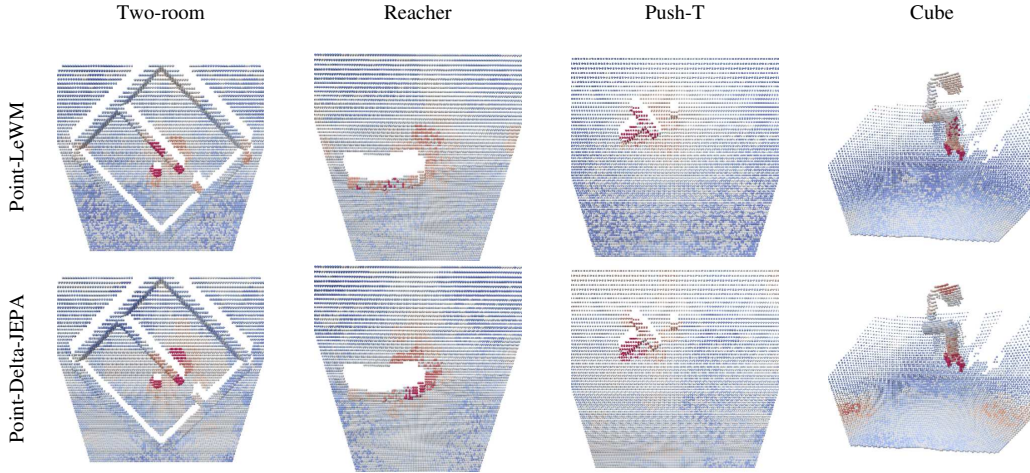


Figure 5: **Encoder attention.** CLS-token attention rollout of the PointViT encoder mapped onto the input points. Warmer colours indicate higher attention, and points no group token covers are omitted. Columns are environments, rows the two models. Attention is normalised per point cloud. Utonia-WM is absent because its frozen PTv3 encoder has no CLS token and attends only locally within serialised patches, so it admits no comparable attention map.

rates behind Table 2. The evaluation seed fixes the same 50 start–goal windows for every model, so any two models are compared on identical episodes and the tests are paired by seed, which removes the seed-to-seed variation in episode difficulty from the comparison.

Two complementary tests answer two different questions. A paired two-sided t -test asks whether the data show any difference at all. A significant result means the two models genuinely differ on the shared episodes, but a non-significant result is not evidence of equality. Positive evidence of parity requires an equivalence test, and we use paired two one-sided tests (TOST): equivalence at a margin of ± 10 points is established ($p < 0.05$) exactly when the data rule out a true difference larger than ten points in either direction. The margin is chosen below the smallest effect the paper interprets, the 13.0-point objective-family gap on Two-Room, so established equivalence is tighter than any gap we reason about. Passing TOST at $p < 0.05$ is equivalent to the 90% confidence interval of the paired difference lying inside the margin. Table 8 reports the more familiar 95% intervals, whose containment in the margin establishes equivalence even at the stricter level $\alpha = 0.025$.

Table 8: Modality comparison of Table 2, Point-LeWM against the re-evaluated image LeWM: difference of means (points) with its 95% confidence interval, paired t -test p , and paired TOST equivalence p at the ± 10 -point margin. Equivalence holds on every benchmark, and the pairing is sensitive enough to resolve the small Push-T deficit.

	Δ (Point-LeWM – LeWM)	95% CI	paired t p	TOST p (± 10)
Two-Room	+1.4	$[-2.0, +4.8]$	0.37	< 0.001
Reacher	−1.6	$[-5.9, +2.7]$	0.42	< 0.001
Push-T	−2.8	$[-5.2, -0.5]$	0.025	< 0.001
OGB-Cube	−3.8	$[-8.1, +0.5]$	0.079	0.005

Table 8 shows the modality comparison. Equivalence holds on every benchmark ($p \leq 0.005$), and Two-Room and Push-T pass even at the stricter ± 5 -point margin, so a modality gap of practical size is ruled out everywhere. Every 95% interval lies inside the ± 10 margin, with the widest edge at -8.1 points on OGB-Cube. In the difference direction, only Push-T is significant, a deficit of 2.8 points ($p = 0.025$) that is real but small next to the image baseline’s 86.4% success. No other benchmark separates the modalities ($p \geq 0.08$), and on Two-Room the point-cloud model attains the higher mean. The objective-family gaps of Section 4 are all confirmed by the same paired test: Point-Delta-JEPA exceeds Point-LeWM by +13.0 points on Two-Room ($p < 10^{-5}$) and +17.4 on OGB-Cube ($p < 10^{-6}$), and Point-LeWM exceeds Point-Delta-JEPA by +12.8 on Push-T ($p < 10^{-3}$). On Reacher the two objectives do not differ (-2.8 , $p = 0.30$), the basis for the main text’s statement that Reacher separates neither objective.

F Broader impacts

This work is foundational research on world-model learning in simulated tabletop environments and is not tied to a particular application or deployment. Its societal effects are those generic to improved robot autonomy. Range-based world models could benefit assistive and industrial robots that must operate where cameras fail, and lower the cost of adapting robots to new tasks. Conversely, more capable autonomous systems can displace labor and could be misused, for example in surveillance platforms that favor range sensors precisely because they work in darkness. The models released here are trained on four simulated benchmarks and pose no direct risk, and we see no mitigations required beyond the norms of open release.